

Indices

Hacer una búsqueda implica que los ítems deben cumplir cierto predicado lógico que se plantea (a nivel query). Si consideramos que se está trabajando con archivos de disco, buscar un ítem, o un conjunto de ítems que cumplan con dicho predicado implica llamar a la operación "Siguierte" hasta que se encuentre un ítem coincidente (en caso de que se sepa de antemano que será el único, por ejemplo, sabiendo que se trata de la clave primaria de la tabla en cuestión), o barrer el archivo completo. Esto implica que, en el peor de los casos, se deberá recorrer el archivo completo en cada búsqueda, conllevando la cantidad de accesos a disco correspondiente a buscar cada bloque de disco que contenga los datos.

Obviamente, se puede plantear ordenar los datos por cierta clave, lo que llevaría a lograr bajar la complejidad promedio de las búsquedas, en caso de que se busque por el atributo por el cual se ordenó el archivo de datos. Las tablas que cumplen esta condición se dicen que tienen un índice agrupado o cluster por el atributo (o la lista de atributos) que les da el ordenamiento. Algo que se puede deducir de esto, es que una tabla sólo puede tener un índice cluster, ya que una disposición física de los datos sólo puede tener un orden. Por lo tanto, la cantidad de índices cluster que puede tener una tabla puede ser 0 o 1, adicionalmente podrá tener una cantidad (acotada por cada DBMS) de índices de otros tipos admitidos (también dependiente del DBMS en cuestión).

Para reducir la complejidad de estas búsquedas sobre los archivos de datos, se plantea mantener estructuras auxiliares que, guardando sólo el atributo de búsqueda, un identificador del ítem y un puntero a su posición en disco permita obtener, sin recorrer el archivo de datos, obtener la posición de el/los ítems de datos buscados.

Los índices consisten (básicamente) en estructuras almacenadas en disco creadas para poder ser utilizadas en memoria principal. Es decir, diseñadas para que el índice, o la parte que se supone que se utilizará esté en memoria, evitando accesos I/O a disco. Se pueden clasificar según varios aspectos; por ejemplo, en cuanto a la cantidad de entradas o valores que existan en el índice, podemos decir si es denso o no denso. El primer caso es cuando hay igual cantidad de entradas en el índice que en el archivo de datos. Esto suele pasar cuando el índice está armado para la clave primaria de la tabla. En el segundo caso, la cantidad es menor a la del archivo de datos.

También se pueden clasificar en agrupados, cuando el orden en el cual están dispuestos los archivos de datos es el del atributo por el cual se construyó el índice, por lo que ambas estructuras estarían ordenadas de igual manera, o no agrupados en caso contrario. Un intento de clasificación puede ser el siguiente:

- **Índices de un nivel:**

- o **Primario:** Se trata de un archivo ordenado, con celdas de tipo (id, puntero al registro). El número de entradas es igual al número de bloques que contienen registros, por lo que es un tipo de índices no denso. La inserción es costosa: esto se debe a que, siendo el índice una estructura lineal ordenada, una inserción en medio de esta implica mover a todos los elementos que le siguen. Sucede lo mismo con el archivo de datos de este tipo.

- o **Agrupamiento:** Ordenados por campo no clave; es decir, el atributo del índice no es la clave principal de la tabla. Muchos registros tienen el mismo valor de clave, ya que la idea del índice es la de poder ejecutar búsquedas por valor de atributo, no por id de ítem de dato. Es un archivo ordenado, con celdas (campo, puntero). Por cada valor almacenado, hay un bloque en disco, por lo que es un índice no denso. Es similar en forma al Hashing, la diferencia radica en que en el verdadero hashing, se trabaja con el id del ítem, mientras que, en el índice por agrupamiento, se trabaja con el valor de un atributo. Es sabido que va a haber sinónimos (o valores iguales), ya que se están construyendo grupos.

o **Secundario:** Es un archivo ordenado, con celdas (campo de no ordenamiento, bloque/registro). Si el índice se construye por campo clave, es denso, ya que los valores serían únicos, en cambio si se lo construye por campo no clave, puede ser de tres formas distintas:

- 1 clave y n entradas, lo que hace al índice denso
- 1 clave y una lista variable con los links a los datos, lo que hace al índice no denso
- Nivel de indirección (el elemento del índice apunta a un contenedor con links a los datos).

- **Índices de múltiples niveles:**

o **Árboles B:** Son estructuras más dinámicas, que permiten operar con inserciones o eliminaciones más fácilmente. A su vez, logran una complejidad logarítmica respecto a la cardinalidad del dominio del atributo por el cual se crea el índice (es importante notar la diferencia de conocer la complejidad de la búsqueda por medio del índice respecto a esta cardinalidad, que respecto a la cardinalidad de la población de datos que exista o pueda existir en la tabla). En los árboles B, todos los nodos tienen punteros que referencian al archivo de datos.

o **Arboles B+:** Son árboles B en donde sólo los nodos de tipo hoja tienen enlaces a páginas de datos. Si el campo es clave, apunta a un bloque/registros. Si no, bloques con apuntadores a los datos. Como los nodos intermedios no tienen links a las páginas, sino sólo links entre nodos de la estructura y claves tienen mayor capacidad para almacenar claves, por lo tanto, mayor grado, luego, su altura es menor. Este tipo de índices admite una operación de carga masiva por la cual cuando tenemos una tabla poblada con muchos registros, es más barato operar en la modalidad de carga masiva, que agregando al índice registro por registro. El algoritmo no será descrito, pero se basa en la posibilidad de ordenar en una lista los atributos de cada registro que será utilizada como nodos hoja y luego ir creando y balanceando los nodos hasta la raíz.

- **Otros tipos de índices:**

o **Bitmaps:** Consisten en arreglos con cadenas de bits que representan, para cada ítem de datos, cuál es el valor del dominio de la clave por la cual se hizo el índice que tiene el atributo. La longitud de estos arreglos es del tamaño de la población (cantidad de ítems), y el tamaño va a depender de la cantidad de valores que pueda tomar el atributo. Suele recomendarse utilizarlo para atributos cuyo dominio tenga una cardinalidad baja, aunque en el DBMS de Oracle se especifica que no es necesario que sea así, ya que puede soportar cantidades medianas de valores, y pueden ser buenos para almacenar claves principales. La gran desventaja es que no tienen buen comportamiento ante inserciones/eliminaciones, ya que tiene que regenerarse o reubicarse parte del índice.

Índices en productos comerciales

Microsoft SQL Server

- Guarda los datos en bloques de 8kB (páginas), basado en el file system de Windows.
- Las filas no pueden exceder esas páginas; restándole headers, links y tamaños, se pueden usar 8060 bytes por registro.
- Los índices y las tablas se guardan en estructuras llamadas externs, que son los bloques mínimos con los que trabaja el I/O de SQL Server. Estos externs son de 8 páginas, es decir, 64kB.
- Los índices principales son agrupados (clustered), los demás, no. Esto es obvio, porque sólo se pueden ordenar físicamente los datos por un atributo en particular).
- Hay dos tipos de tablas:
 - **Clustered:** Archivo ordenado por la clave. El índice es un árbol B con páginas que forman listas enlazadas por nivel.
 - **Heaps:** sin orden, ni índice. Puede tener índices no agrupados (clustered).
 - Además del índice clustered, se le pueden agregar 249 índices, que son también árboles B.
 - Los tipos de datos Text, NText e imágenes van a un único storage por tabla, llamado colección.

Oracle

- Árbol B
- Bitmap
- Partitioned indexes
- Function-based indexes – basado en expresiones
- Domain indexes

MySQL

Todos los índices en InnoDB son árboles binarios (B-trees) donde las entradas de índice se almacenan en páginas que son las hojas del árbol. El tamaño predeterminado de una página de índice es 16KB.

Si las entradas del índice se insertan en un orden secuencial (sea ascendente o descendente), las páginas de índice se llenarán en aproximadamente 15/16 de su capacidad. Si las entradas se insertan en un orden aleatorio, las páginas se llenarán entre 1/2 y 15/16 de su capacidad.

Posee índices de hash para tablas que entren en memoria principal.

Dependiendo del Motor de almacenamiento de MySQL, son los tipos de índice que están permitidos, a continuación, se los ilustra en una tabla:

MyISAM	Árbol B (BTREE)
InnoDB	Árbol B (BTREE)
MEMORY/HEAP	Árbol B (BTREE) y Hash

Bibliografía y referencias

- "Sistemas de Bases de Datos - Conceptos fundamentales" - Elmasri/Navathe - Segunda Edición Addison Wesley Iberoamericana.
- "Sistemas de Gestión de Bases de Datos" - Ramakrishnan/Gerke - Tercera edición Ed. McGraw Hill
- "Introducción a las bases de datos Relacionales" - Mendelzohn/Ale - Pearson Education